

UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

Programación Avanzada

IMPERIOS EN GUERRA

Informe técnico del proyecto final

Videojuego de estrategia en tiempo real en C# y Unity con patrón MVC, concurrencia y manejo de archivos

Docente: Nancy Yaneth Gelvez García

Integrantes:

Juan Andres Villada Saray 20242020261

William Stiven Pulecio Murcia 20252020115

Repositorio: [AndresVillada/ImperiosEnGuerra: RTS estilo Age of Empires en C#/Unity](#)

Bogotá D. C., septiembre de 2026

1. Introducción

1.1 Contexto y objetivo

«Imperios en Guerra» es un videojuego de estrategia en tiempo real (RTS) inspirado en Age of Empires, desarrollado como proyecto final de la asignatura Programación Avanzada. Se implementó en C# sobre el motor Unity, como aplicación de escritorio, con el propósito de aplicar de manera efectiva tres conceptos: el patrón de diseño Modelo-Vista-Controlador (MVC), la concurrencia mediante hilos y la comunicación en red.

En el juego, el jugador humano dirige a Grecia, que debe conquistar el territorio de un rival controlado por la máquina, en un único mapa compartido de 20×20 celdas. Para lograrlo debe administrar cuatro recursos, construir edificios, entrenar tropas y disputar con el rival los depósitos de recursos repartidos por el mapa.

Este informe explica cómo se emplearon el patrón MVC y la concurrencia, deja constancia de que la comunicación en red no fue implementada y por qué, describe los archivos que genera el programa, presenta los diagramas de clases y de flujo que ilustran la arquitectura, y documenta las pruebas de escritorio realizadas sobre las secciones más críticas del sistema: concurrencia, ataques y condición de victoria.

1.2 Alcance y decisiones de diseño del proyecto

Durante el desarrollo se tomaron las siguientes decisiones, algunas por indicación de la docente y otras por razones técnicas o de tiempo:

Tabla 1. Decisiones de diseño y su justificación.

Decisión	Justificación
Un solo mapa compartido de 20×20 , en lugar de un mapa por jugador	Permite que los dos bandos compitan por los mismos depósitos de recursos, que es donde se demuestra la concurrencia. La dimensión coincide con el Tilemap dibujado en Unity.
Jugador humano contra una IA, sin segundo jugador humano	La docente indicó que la comunicación en red es opcional. Un rival controlado por la máquina permite jugar y probar el sistema completo en una sola instancia.
Cuatro recursos: oro, madera, piedra y metal	La guía menciona tres recursos (oro, madera y comida). Se sustituyó la comida por piedra y metal para dar usos distintos a edificios, tropas y defensas.
Los hilos se implementan dentro del Modelo, no en el Controlador	Indicación expresa de la docente: el Controlador solo transporta información entre el Modelo y la Vista, sin lógica propia.
Tres tipos de tropa (espadachín, piquero y arquero) y un aldeano	Se ajustó el plantel al arte disponible (paquetes de sprites libres) sin perder variedad táctica: cuerpo a cuerpo ligero, cuerpo a cuerpo pesado y a distancia.
La IA rival vive en el Modelo	Tomar decisiones es lógica de juego. La IA se ejecuta en dos hilos propios: uno de economía y uno militar.

1.3 Herramientas y entorno

Tabla 2. Herramientas utilizadas.

Elemento	Detalle
Motor	Unity 6 (6000.6.0f1), plantilla Universal 2D (URP 2D)
Lenguaje	C# (versión 9, la del compilador de Unity 6)
Editor de código	Visual Studio Code
Control de versiones	Git, con repositorio compartido en GitHub (trabajo en pareja)
Arte	Paquetes «Medieval RTS» (Kenney) y «Tiny Swords», ambos de licencia CC0
Plataforma objetivo	Aplicación de escritorio (Windows, macOS o Linux); no se usa consola ni WebGL

1.4 Estructura del documento

La sección 2 describe el juego; la 3, la arquitectura MVC; la 4, la concurrencia; la 5, la comunicación en red; la 6, el manejo de archivos; la 7 reúne los diagramas; la 8 documenta las pruebas de escritorio; y la 9 presenta las conclusiones, el cumplimiento de los requisitos y las limitaciones.

2. Descripción funcional del juego

2.1 Mapa, bandos y recursos

El mapa es una matriz de 20×20 celdas compartida por ambos bandos. Grecia, el jugador humano, inicia con su Centro Urbano en la celda (2, 2); el rival, llamado «El Resto» y controlado por la máquina, en la celda (17, 17). Cada bando comienza con tres aldeanos y con 100 unidades de cada recurso.

El mapa contiene 22 depósitos de recursos repartidos de forma casi simétrica: algunos cerca de cada base y otros en las bandas laterales y en el centro, que es la zona disputada. Cada depósito es pequeño, de modo que se agota y los aldeanos deben buscar el siguiente.

Tabla 3. Depósitos de recursos del mapa.

Recurso	Depósitos	Unidades por depósito	Usos principales
Madera	8	200	Casas, Taller y Piquero
Oro	6	250	Todas las tropas
Piedra	4	150	Torre
Metal (hierro)	4	150	Torre

2.2 Unidades y edificios

Tabla 4. Unidades.

Unidad	Vida	Daño	Rango	Costo
Aldeano	25	—	—	Se reciben 3 al inicio de cada bando; recolecta 5 unidades por segundo
Espadachín	50	10	1	30 de Oro
Piquero	70	7	1	20 de Oro y 15 de Madera
Arquero	30	8	3	20 de Oro

Tabla 5. Edificios.

Edificio	Vida	Costo	Función
Centro Urbano	500	Existe desde el inicio	Si se destruye, el bando pierde. Desde él se construyen los demás edificios.
Casa	100	80 de Madera	Otorga población (valor definido en el Modelo; el tope de población no se aplica).
Taller	200	120 de Madera	Entrena tropas.
Torre (Defensa)	250	80 de Piedra y 40 de Metal	Dispara sola a los enemigos en su rango de 3 celdas (daño 8, un disparo cada 2 s).

La construcción avanza 10 % cada 500 ms (unos 5 s en total) y el entrenamiento de una tropa tarda 3 s. Entre dos entrenamientos de un mismo jugador debe transcurrir un enfriamiento de 8 s para Grecia y de 10 s para la IA.

2.3 Acciones del jugador y controles

Tabla 6. Acciones del jugador y quién las valida.

Acción	Cómo se ejecuta	Validación (en el Modelo)
Seleccionar	Clic izquierdo sobre un edificio o unidad propios	<code>Jugador.EsPropio</code>
Construir	Con el Centro Urbano seleccionado: teclas 1 (Casa), 2 (Taller) o 3 (Torre)	<code>Jugador.ConstruirCerca</code> : constructor propio, celda libre y recursos suficientes
Entrenar	Con un Taller seleccionado: teclas 1 (Espadachín), 2 (Piquero) o 3 (Arquero)	<code>Jugador.EntrenarTropaCerca</code> : Taller propio y listo, enfriamiento y recursos
Mover	Con una unidad seleccionada, clic en una celda vacía	<code>Jugador.OrdenarMovimiento</code> : unidad propia y viva, destino dentro del mapa
Recolectar	Con un aldeano seleccionado, clic sobre un depósito	<code>Jugador.OrdenarRecoleccion</code> : aldeano propio y depósito disponible
Atacar	Con una tropa seleccionada, clic sobre una unidad o un edificio rival	<code>Jugador.OrdenarAtaque</code> : tropa propia, rango y vida

Además, las tropas de Grecia atacan automáticamente lo que tengan dentro de su rango (defensa automática), de modo que el jugador no tenga que ordenar cada golpe.

2.4 Inteligencia artificial rival

La IA usa los mismos métodos públicos de `Jugador` que emplea el humano, sin atajos especiales. Corre en dos hilos. El ciclo de economía, cada 2 s, construye su Taller y hasta cuatro Casas cerca de su Centro Urbano y entrena una tropa cada 10 s, con un máximo de seis tropas vivas. El ciclo militar, cada 1 s, espera a reunir una oleada de al menos cuatro tropas; desde entonces cada tropa avanza hacia el objetivo más cercano y lo ataca cuando lo tiene en su rango, con un golpe cada 2 s.

2.5 Condiciones de victoria y derrota

Un jugador queda derrotado si se cumple cualquiera de estas dos condiciones, el «y/o» del enunciado: (a) su Centro Urbano fue destruido, o (b) todas las unidades militares que alguna vez tuvo están destruidas. Si un jugador nunca entrenó tropas no se considera derrotado por la segunda condición. Si ambos quedan derrotados a la vez, la partida termina en empate. El resultado se muestra en la interfaz con un panel de victoria o de derrota.

3. Arquitectura MVC

3.1 Criterio de separación de responsabilidades

El código se organizó en tres carpetas y tres espacios de nombres, `ImperiosEnGuerra.Modelo`, `ImperiosEnGuerra.Vista` e `ImperiosEnGuerra.Controlador`, bajo `Assets/Scripts/`. Para decidir a qué capa pertenece cada pieza se aplicó un único criterio: **si una interfaz distinta, por ejemplo una de consola, tendría que volver a escribir esa lógica, la lógica pertenece al Modelo; si solo tiene sentido para esta interfaz gráfica, es de presentación y puede permanecer en la Vista o en el Controlador.**

3.2 Responsabilidades de cada capa

Tabla 7. Responsabilidades y restricciones de cada capa.

Capa	Clases	Responsabilidad	Restricciones
Modelo	<code>Partida</code> , <code>Jugador</code> , <code>JugadorIA</code> , <code>Mapa</code> , <code>Unidad</code> y subclases, <code>Edificio</code> y subclases, <code>Recurso</code> , <code>TipoRecurso</code> , <code>ConfiguradorPartida</code> , <code>GestorArchivos</code>	Estado y reglas del juego, concurrencia, IA y persistencia en archivos	Clases C# planas (POCO). No hace ninguna referencia a <code>UnityEngine</code>
Vista	<code>MapView</code> , <code>UnidadView</code> , <code>EdificioView</code> , <code>HUDController</code> , <code>FinPartidaView</code> , <code>EntrenamientoCooldownView</code>	Mostrar el estado: sprites, barras de vida, HUD, paneles y animaciones	Solo lee el Modelo: no lo modifica ni decide reglas
Controlador	<code>GameController</code> , <code>SincronizacionVistaController</code> , <code>InputController</code>	Transportar información: entrada del usuario hacia órdenes al Modelo, y estado del Modelo hacia la Vista	Sin lógica de negocio propia. Solo <code>GameController</code> es un <code>MonoBehaviour</code>

El Modelo no conoce a la Vista ni al Controlador. Cuando algo debe notificarse sin acoplar las capas se usan eventos de C# (`Murio en Unidad` y `Destruido en Edificio`) y colas de eventos (`ConcurrentQueue<EventoJuego>`) que consume el gestor de archivos. La Vista obtiene los datos consultando el Modelo en cada `Update()`, es decir, por sondeo; el Modelo nunca llama a la interfaz.

3.3 Flujo de información entre capas

La Figura 5 muestra las clases de cada capa y sus dependencias, y la Figura 1 el flujo general de ejecución. En síntesis: (1) `GameController` pide al Modelo que arme la partida (`ConfiguradorPartida`) y crea los otros dos controladores; (2) en cada cuadro, `SincronizacionVistaController` recorre el estado del Modelo y crea o actualiza las vistas (Modelo hacia Vista); (3) `InputController` convierte clics y teclas en una llamada a un método de `Jugador` (Controlador hacia Modelo), que valida y ejecuta la orden; y (4) los hilos del Modelo cambian el estado en paralelo y la Vista lo refleja en el siguiente cuadro.

La Figura 6 ilustra un ejemplo completo: el jugador selecciona su Centro Urbano y pulsa la tecla 2. El Controlador solo traduce la entrada; el Modelo decide si la orden es válida (constructor propio, celda libre y recursos suficientes), lanza el hilo que construye y registra los eventos; la Vista crea el edificio y su barra de progreso.

3.4 Auditoría de la separación de responsabilidades

Al terminar la primera versión funcional se auditó el código con el criterio anterior y se encontró lógica de negocio ubicada en la Vista o en el Controlador. Todas las piezas se trasladaron al Modelo:

Tabla 8. Lógica de negocio encontrada fuera del Modelo y su corrección.

Hallazgo (ubicación original)	Por qué es lógica de negocio	Corrección
Las Vistas calculaban el porcentaje de vida y de construcción	Cualquier interfaz que muestre la vida lo necesita	<code>Unidad.PorcentajeVida</code> , <code>Edificio.PorcentajeVida</code> y <code>Edificio.FraccionConstruccion</code> ; las Vistas solo aplican el valor
La Vista decidía si «ya se puede entrenar» con un umbral	Es la regla del enfriamiento	<code>Jugador.EntrenamientoDisponible</code> ; la Vista solo muestra la respuesta
El Controlador arrancaba el disparo de las Torres cuando aparecía su vista	Regla del juego atada a que la interfaz «vea» la torre	<code>Jugador.ConstruirEdificio</code> activa la defensa al construirla
El Controlador creaba edificios y tropas (<code>new House</code> , <code>new Espadachin</code>) y elegía la celda	Decide cómo se construye y dónde	<code>Jugador.ConstruirCerca</code> y <code>EntrenarTropaCerca</code> , con los enumerados <code>TipoEdificio</code> y <code>TipoTropa</code> y <code>FabricaDeEntidades</code>

Hallazgo (ubicación original)	Por qué es lógica de negocio	Corrección
El Controlador fabricaba los eventos de ataque y validaba rangos y límites del mapa	Las validaciones y el registro son del Modelo	<code>Jugador.OrdenarAtaque y OrdenarMovimiento</code>
El Controlador decidía qué unidad es «propia»	Regla de pertenencia	<code>Jugador.EsPropio</code>
El Controlador comparaba nombres para elegir entre victoria y derrota	Interpreta el resultado de la partida	<code>Partida.GanoJugador</code>
El Controlador ordenaba guardar <code>resultado_final.txt</code>	Persistencia	<code>GestorArchivos</code> lo guarda al ver la partida finalizada
El Controlador contenía posiciones iniciales, depósitos y aldeanos	Es el estado inicial del juego	<code>ConfiguradorPartida (Modelo)</code>
El Controlador buscaba celdas libres	Consulta sobre el estado del mapa	<code>Mapa.BuscarCeldaLibreCercana</code>

3.5 Qué permanece legítimamente en la Vista y el Controlador

- **Vista:** elegir el sprite según el tipo de unidad o edificio, teñir de rojo las piezas de la IA, la transparencia de un edificio mientras se construye, la animación de desvanecimiento al morir, el deslizamiento entre celdas, y el color y formato del indicador de enfriamiento.
- **Controlador:** convertir la posición del mouse en una celda, recordar qué está seleccionado, asignar teclas a acciones, traducir el tipo de una entidad del Modelo a su prefab, y coordinar el arranque y el cierre.

3.6 Verificación objetiva

La separación se comprobó de forma objetiva: el Modelo completo se compila de manera independiente, sin motor gráfico, con el compilador de C# 9, la misma versión del lenguaje que usa Unity 6, y las pruebas de la sección 8 se ejecutan sobre ese Modelo. Si el Modelo dependiera de Unity, esa compilación no sería posible. En total, la solución tiene 19 archivos de código y unas 3.700 líneas, comentarios incluidos: aproximadamente 2.600 en el Modelo, 540 en la Vista y 580 en el Controlador.

4. Concurrencia

4.1 Objetivo y modelo de hilos

La concurrencia se usa para que las actividades del juego que duran en el tiempo (recolectar, caminar, construir, entrenar, disparar, decidir de la IA y registrar eventos) avancen en paralelo sin bloquear la interfaz. Cada una corre en su propia tarea (`Task.Run`) dentro del Modelo, como indicó la docente.

Dos reglas guían el diseño. La primera: la API de Unity solo puede invocarse desde el hilo principal; por eso el Modelo no referencia UnityEngine y los hilos secundarios nunca tocan objetos de la escena. Los hilos cambian el estado del Modelo, que usa colecciones seguras para hilos, y el hilo principal lo lee en cada `Update()` para actualizar la Vista. La segunda: todas las esperas se hacen con `await Task.Delay(...)` y no con `Thread.Sleep`, de modo que mientras una actividad espera no ocupa un hilo del grupo de hilos (ThreadPool); en la primera versión cada `Sleep` dejaba un hilo bloqueado sin trabajar.

4.2 Inventario de hilos

Tabla 9. Hilos del sistema, su sincronización y su terminación.

Hilo	Cantidad	Propósito	Sincronización	Terminación
Recolección	1 por aldeano	Elegir un depósito, caminar hasta él (un paso cada 400 ms), recolectar 5 unidades por segundo y pasar al siguiente cuando se agota	<code>Recurso.Extraer (lock)</code> ; <code>AgregarRecurso (AddOrUpdate atómico)</code> ; Mapa (lock de celdas)	Aldeano muerto, sin depósitos de su tipo, orden nueva o <code>DetenerHilos</code>
Construcción	1 por edificio	Pagar, reservar la celda y avanzar 10 % cada 500 ms	<code>PagarCosto (lock)</code> ; Mapa (lock); <code>Edificio.AvanzarConstruccion (lock)</code>	Al llegar a 100 % o <code>DetenerHilos</code>
Entrenamiento	1 por tropa	Pagar, esperar 3 s y desplegar en una celda libre	<code>PagarCosto</code> ; Mapa. <code>TryColocarUnidadCerca (atómico)</code>	Al desplegar o <code>DetenerHilos</code>
Movimiento	1 por orden	Caminar una celda cada 300 ms	Mapa. <code>MoverUnidad (lock)</code>	Llega, se bloquea, muere, orden nueva o <code>DetenerHilos</code>
Torre	1 por torre	Disparar cada 2 s al rival en rango	<code>RecibirDanio (lock por instancia)</code>	Torre destruida o <code>DetenerHilos</code>
Defensa de tropas de Grecia	1	Cada 1 s, cada tropa ataca lo que tenga en su rango	<code>RecibirDanio (lock por instancia)</code>	<code>DetenerHilos</code>
IA: economía	1	Cada 2 s decide construir o entrenar	<code>Jugador.EntrenarTropa (lock del enfriamiento)</code> ; Random con lock	<code>JugadorIA.Detener</code> o fin de la partida
IA: militar	1	Cada 1 s mueve y ataca con sus tropas	Mapa. <code>MoverUnidad</code> ; Random con lock	<code>JugadorIA.Detener</code> o fin de la partida
Log	1	Cada 300 ms vacía las colas de eventos y escribe <code>log_partida.txt</code>	<code>ConcurrentQueue</code> ; lock de escritura	Cancelación de su token o fin de la partida, con un último vaciado
Principal (Unity)	1	<code>Update</code> : lee el Modelo, actualiza la Vista y lee las entradas del jugador	Solo lectura de colecciones concurrentes	—

4.3 Recursos compartidos y mecanismos de sincronización

Tabla 10. Recursos compartidos, mecanismo de protección y riesgo que se evita.

Recurso compartido	Mecanismo	Riesgo que evita
Recurso.Cantidad	lock sobre un objeto privado en Extraer y EstaAgotado	Que dos aldeanos lean «quedan 10» y ambos resten 8, dejando el depósito en -6
Jugador.Recursos	ConcurrentDictionary con AddOrUpdate (AgregarRecurso) y lock en PagarCosto	Perder sumas simultáneas; pagar dos veces con el mismo saldo
Enfriamiento de entrenamiento	lock dedicado sobre la hora del próximo entrenamiento permitido	Aceptar dos órdenes dentro del enfriamiento (la IA y el hilo principal lo consultan a la vez)
Mapa: matriz de celdas	lock sobre la matriz (los arreglos de .NET no son seguros para hilos) y diccionarios concurrentes	Corrupción de la matriz; dos unidades en una misma celda
Colocación de tropas	Buscar la celda libre y ocuparla dentro del mismo lock (TryColocarUnidadCerca)	Que las cinco tropas de un lote queden apiladas en una celda
Movimiento	MoverUnidad completo dentro de un lock; rechaza unidades muertas	Que una unidad que muere al caminar «resucite» su celda
Retiro de un depósito agotado	TryRetirarRecursoAgotado, atómico	Que dos aldeanos anuncien el mismo agotamiento
Vida y progreso de construcción	lock por instancia en Unidad y Edificio	Daño y avance de obra simultáneos sobre el mismo objeto
Estado de la Partida	lock; método idempotente	Ganador inconsistente si varios hilos verifican a la vez
Colecciones de Jugador	ConcurrentDictionary y ConcurrentQueue	Corrupción al leer y escribir desde varios hilos
Generador aleatorio	Acceso a través de un método con lock	System.Random no es seguro para hilos y puede degenerar en valores constantes
Archivo de log	lock alrededor de la escritura	Líneas entrelazadas
Orden activa de cada unidad	ConcurrentDictionary con AddOrUpdate y tokens ligados	Que un aldeano camine a un depósito y a una celda a la vez

4.4 Orden de candados y prevención de interbloqueos

Cuando un método necesita más de un candado, los toma siempre en el mismo orden: primero el candado de las celdas del Mapa y después el candado de la entidad (MoverUnidad y TryColocarUnidadCerca llaman a Unidad.MoverA con el candado de celdas tomado). El camino contrario no existe: Unidad.RecibirDanio y Edificio.RecibirDanio liberan su propio candado **antes** de disparar los eventos Murio y Destruido, cuyo manejador, en Mapa, toma el candado de celdas. Así no puede formarse un ciclo de espera.

4.5 Finalización ordenada de los hilos

Ningún hilo se detiene a la fuerza: todos observan un CancellationToken. Cada Jugador posee una fuente de cancelación de la que cuelgan sus hilos (DetenerHilos); cada unidad tiene además una orden activa con un token ligado al general, de modo que una orden nueva cancela la anterior; la IA y el hilo del log tienen sus propias fuentes. Al terminar la partida o al cerrar el juego,

`GameController.DetenerHilos` cancela todas. Cada `Task.Delay` pendiente lanza `OperationCanceledException`, que el método `Jugador.Lanzar` trata como fin normal; cualquier otra excepción se registra como evento de error en lugar de matar el hilo en silencio.

4.6 Caso de estudio: la competencia por los recursos

La competencia entre Grecia y la IA es el ejemplo central de concurrencia del proyecto. Los dos bandos comparten los depósitos: sus aldeanos, cada uno en su hilo, eligen el depósito más cercano de su tipo, caminan hasta quedar junto a él y recolectan. Si dos aldeanos rivales llegan al mismo árbol, el `lock` de `Recurso.Extraer` decide quién se lleva cada unidad; el que llega tarde lo registra («llegó tarde: el depósito ya lo agotó otro aldeano») y se redirige al siguiente depósito más cercano. La Figura 7 muestra el ciclo del hilo del aldeano.

El jugador puede intervenir: al seleccionar un aldeano y hacer clic sobre un depósito, `Jugador.OrdenarRecoleccion` cancela su orden anterior y lo envía allí; cuando ese depósito se agota, sigue solo con el más cercano del mismo tipo. Las pruebas PE-C1 y PE-C2 (sección 8) verifican la propiedad esencial: lo recolectado entre todos los aldeanos suma exactamente lo que había en los depósitos, sin pérdidas ni excesos. La prueba PE-C3 verifica lo mismo para el pago de costos con diez hilos simultáneos durante cincuenta rondas.

4.7 La interfaz no se congela

Como todas las actividades largas esperan con `Task.Delay` fuera del hilo principal y el hilo principal solo lee el estado, la ventana de Unity nunca se bloquea. La construcción, el entrenamiento o el disparo de una Torre avanzan mientras el jugador sigue dando órdenes.

5. Comunicación en red

5.1 Estado

La comunicación en red no fue implementada. La docente indicó que este requisito pasó a ser opcional, y el proyecto se orientó a un jugador humano contra una IA que corre en la misma instancia. Por ello el juego no usa sockets, WebSockets, REST ni SOAP.

5.2 Decisiones de diseño que la facilitarían

- Todas las acciones del jugador pasan por un conjunto pequeño y explícito de métodos de `Jugador` (`ConstruirCerca`, `EntrenarTropaCerca`, `OrdenarMovimiento`, `OrdenarRecoleccion` y `OrdenarAtaque`), que son candidatos naturales a mensajes de red.
- `EventoJuego` ya es un mensaje simple (tipo y texto) y las colas de eventos ya son seguras para hilos.

- El Modelo no depende de Unity, de modo que puede ejecutarse en un servidor sin interfaz.

5.3 Propuesta (no implementada)

Como trabajo futuro se propone una conexión TCP (`System.Net.Sockets`) entre dos instancias, con mensajes en JSON del tipo `{ accion, jugador, parametros }`. Un hilo de escucha por conexión, basado en `Task` y `await`, deposita los mensajes recibidos en una `ConcurrentQueue` y el Modelo los aplica llamando a los mismos métodos `Ordenar...`, de modo que las reglas y validaciones no se dupliquen. La pérdida de conexión se detectaría por excepciones de entrada/salida o por un tiempo de espera, se registraría como evento y se notificaría a la interfaz con un panel. Esta descripción es una propuesta de diseño y no forma parte del sistema entregado.

6. Manejo de archivos

El Modelo genera tres archivos de texto en el directorio de ejecución mediante `System.IO`, a cargo de la clase `GestorArchivos`:

Tabla 11. Archivos generados por el programa.

Archivo	Cuándo se genera	Contenido
<code>configuracion.txt</code>	Una vez, al iniciar la partida	Nombre de los dos jugadores, dimensiones del mapa y fecha
<code>log_partida.txt</code>	Durante la partida; el hilo del log vacía las colas de eventos cada 300 ms	Cada evento relevante con el formato de la guía: Turno, Acción y Resultado
<code>resultado_final.txt</code>	Una sola vez, cuando la partida queda finalizada	Ganador y fecha de finalización

6.1 Decisiones de diseño

- Los métodos del Modelo no escriben en el archivo: solo encolan un `EventoJuego`. Un hilo independiente los escribe, de modo que un evento nuevo queda registrado sin modificar `GestorArchivos`.
- Al iniciar una partida se borran el log y el resultado anteriores: el log se escribe en modo «agregar» y, sin esto, acumulaba varias partidas.
- Al cancelar el hilo del log se hace un último vaciado de las colas, para no perder el evento final, por ejemplo el golpe que destruye el Centro Urbano.
- `resultado_final.txt` lo escribe el propio `GestorArchivos` al ver la partida finalizada; si el juego se cierra con la partida en curso no se escribe ningún resultado. Un error de entrada/salida, por ejemplo un archivo abierto por otro programa, se captura para no detener el hilo.

6.2 Ejemplo de archivos generados

A continuación se muestra un ejemplo generado por el sistema en una ejecución de verificación del Modelo sin interfaz gráfica: 45 s de partida con la IA activa y, al final, la caída del Centro Urbano de la IA. La fecha depende de la configuración regional del equipo.

configuracion.txt

```
Jugador 1: Grecia - Mapa 20x20
Jugador 2: El Resto - Mapa 20x20
Fecha: 09/28/2026 04:40:46
```

log_partida.txt (primeros y últimos eventos)

```
Turno: Grecia
Acción: Recoleccion
Resultado: Aldeano se dirige a recolectar Madera en (5,4).

Turno: Grecia
Acción: Recoleccion
Resultado: Aldeano se dirige a recolectar Oro en (8,3).

Turno: Grecia
Acción: Recoleccion
Resultado: Aldeano se dirige a recolectar Piedra en (6,9).

Turno: El Resto
Acción: Recoleccion
Resultado: Aldeano se dirige a recolectar Madera en (14,15).
...
Turno: El Resto
Acción: Entrenamiento
Resultado: Piquero entrenado y desplegado en (16,15).

Turno: El Resto
Acción: Ataque
Resultado: La IA lanza una oleada con 4 tropas.

Turno: Sistema
Acción: FinDePartida
Resultado: Resultado final: Grecia
```

resultado_final.txt

```
Ganador: Grecia
Fecha de finalización: 09/28/2026 04:41:31
```

El log de esa ejecución tiene 255 eventos (1.020 líneas): 199 de recolección, 50 de construcción, 4 de entrenamiento, 1 de ataque y 1 de fin de partida.

7. Diagramas

Los diagramas se generaron a partir de los miembros reales del código. En los diagramas de flujo, los colores indican la capa o el tipo de actividad: verde para el Modelo, amarillo para la Vista, azul para el Controlador y violeta para un hilo (Task); los rectángulos de doble borde son subprocessos, y los círculos son conectores entre diagramas.

7.1 Flujo general de la aplicación

La Figura 1 muestra la ejecución de una partida: el arranque (Start), la configuración del Modelo con sus hilos, el ciclo de Update en el que se verifica el ganador, se sincroniza la Vista y se leen las órdenes del jugador, y el cierre con la detención de los hilos y el guardado del resultado.

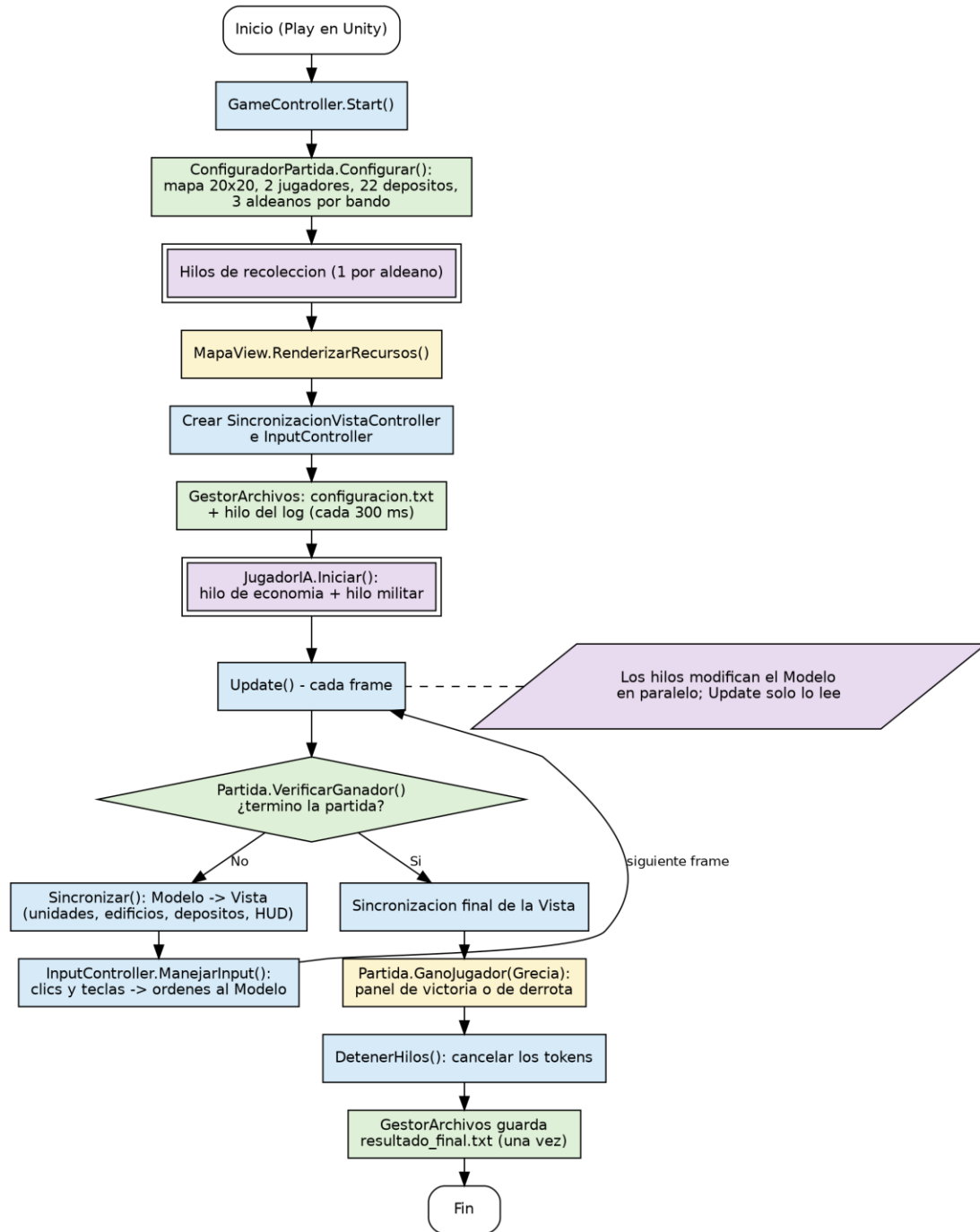


Figura 1. Diagrama de flujo de la ejecución general de la aplicación (una partida).

7.2 Diagramas de clases del Modelo

Las Figuras 2, 3 y 4 presentan el Modelo en tres partes: las jerarquías de unidades y edificios, el núcleo (partida, jugador, mapa y recurso) y los servicios (configuración, IA, persistencia y fábrica). Se usa notación UML: la herencia se dibuja con un triángulo hueco, la realización de una interfaz con línea discontinua y triángulo hueco, la composición con un rombo negro, la agregación con un rombo blanco, la asociación con una flecha abierta y la dependencia con una flecha discontinua. Los miembros marcados con `{lock}` están protegidos con un candado.

Clases fundamentales. No todas las clases tienen el mismo peso en el funcionamiento del programa. Se seleccionaron como fundamentales las que se describen en la tabla siguiente, por su papel en la arquitectura, la concurrencia o las reglas del juego.

Tabla 12. Clases fundamentales y justificación.

Clase	Por qué es fundamental
Jugador	Concentra las acciones del juego y la creación de los hilos de cada bando; es la única puerta de entrada de las órdenes del jugador y de la IA.
Mapa	Es el estado compartido más disputado: matriz de celdas y ubicación de depósitos, edificios y unidades. Centraliza las búsquedas y los movimientos bajo un mismo candado.
Recurso	Es lo que se disputa: su <code>lock</code> garantiza que la extracción concurrente sea correcta.
Unidad y Edificio	Definen la vida, el daño y la construcción, y los eventos <code>Murio</code> y <code>Destruído</code> con los que el Mapa libera celdas.
Partida	Evalúa las condiciones de victoria y derrota de forma idempotente y segura para hilos.
JugadorIA	Implementa el rival en dos hilos propios y demuestra que la IA usa las mismas reglas que el humano.
ConfiguradorPartida	Concentra el estado inicial (mapa, depósitos y aldeanos), que es una regla del juego.
GestorArchivos	Aísla la persistencia y desacopla el registro de eventos con un hilo propio.
GameController	Orquesta el arranque y el cierre; es el único <code>MonoBehaviour</code> del Controlador.
SincronizacionVistaController e InputController	Son los dos puentes: del Modelo hacia la Vista, y del jugador hacia el Modelo.

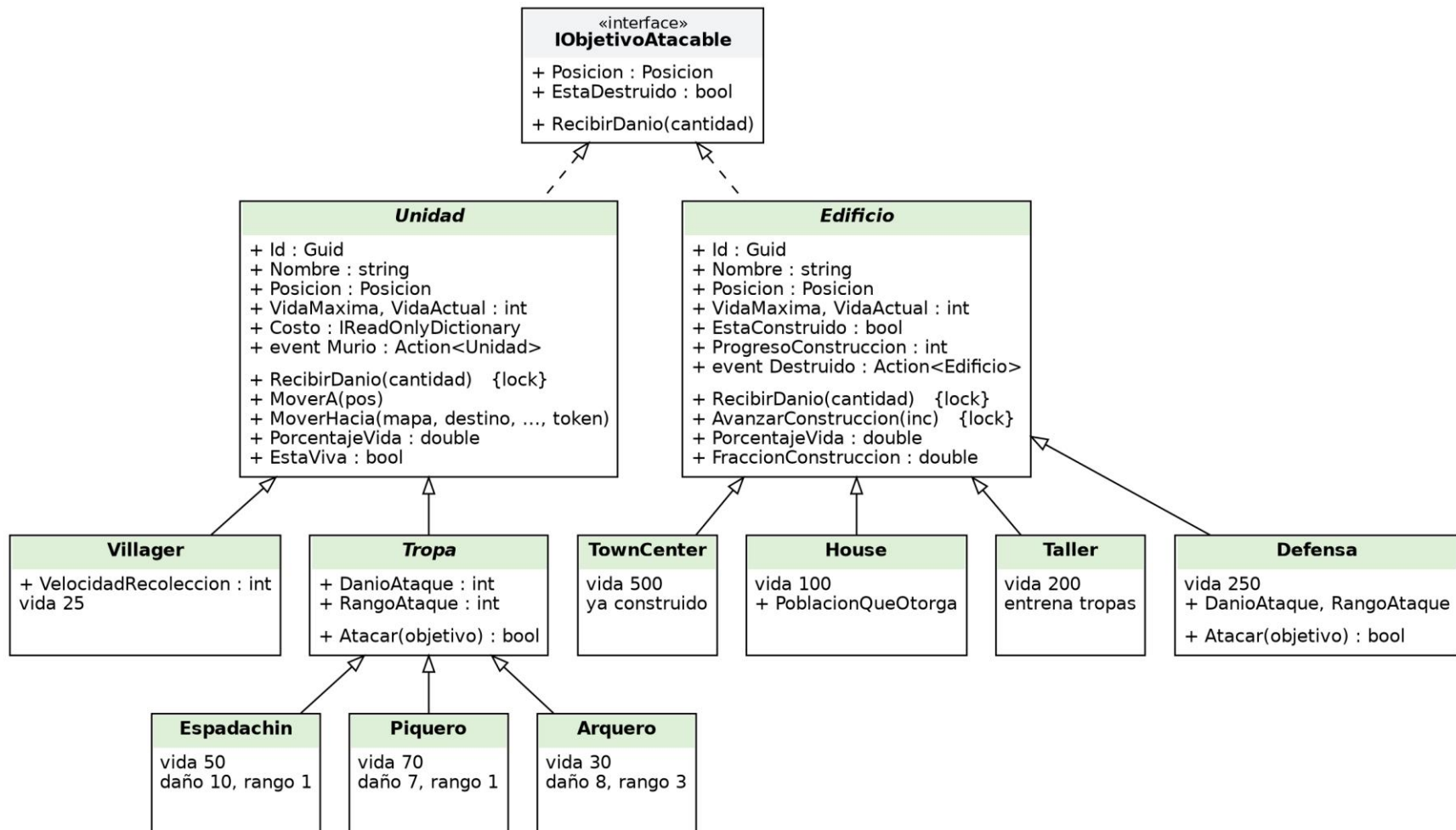


Figura 2. Diagrama de clases del Modelo: jerarquías de unidades y edificios.

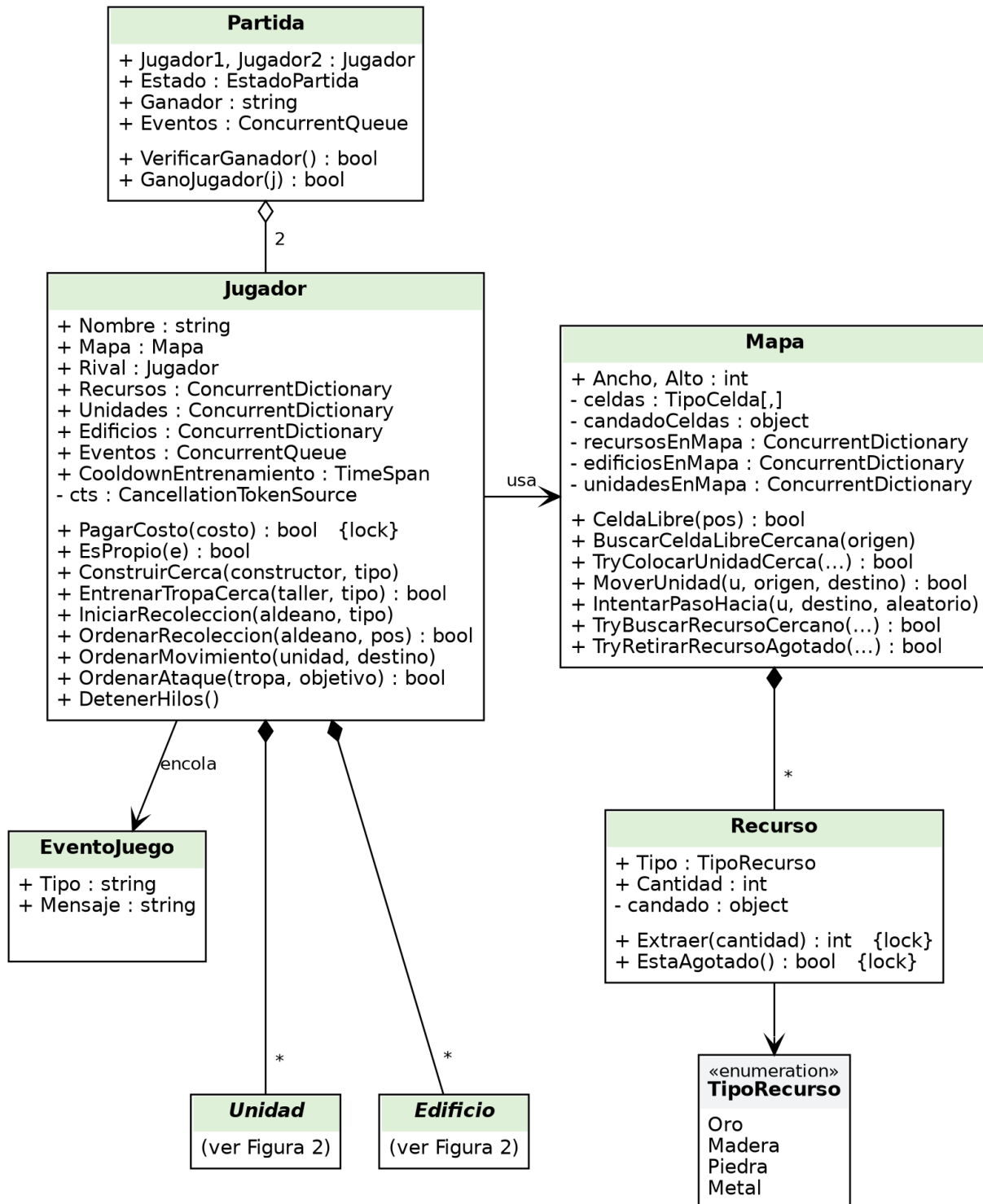


Figura 3. Diagrama de clases del Modelo: núcleo (partida, jugador, mapa y recurso).

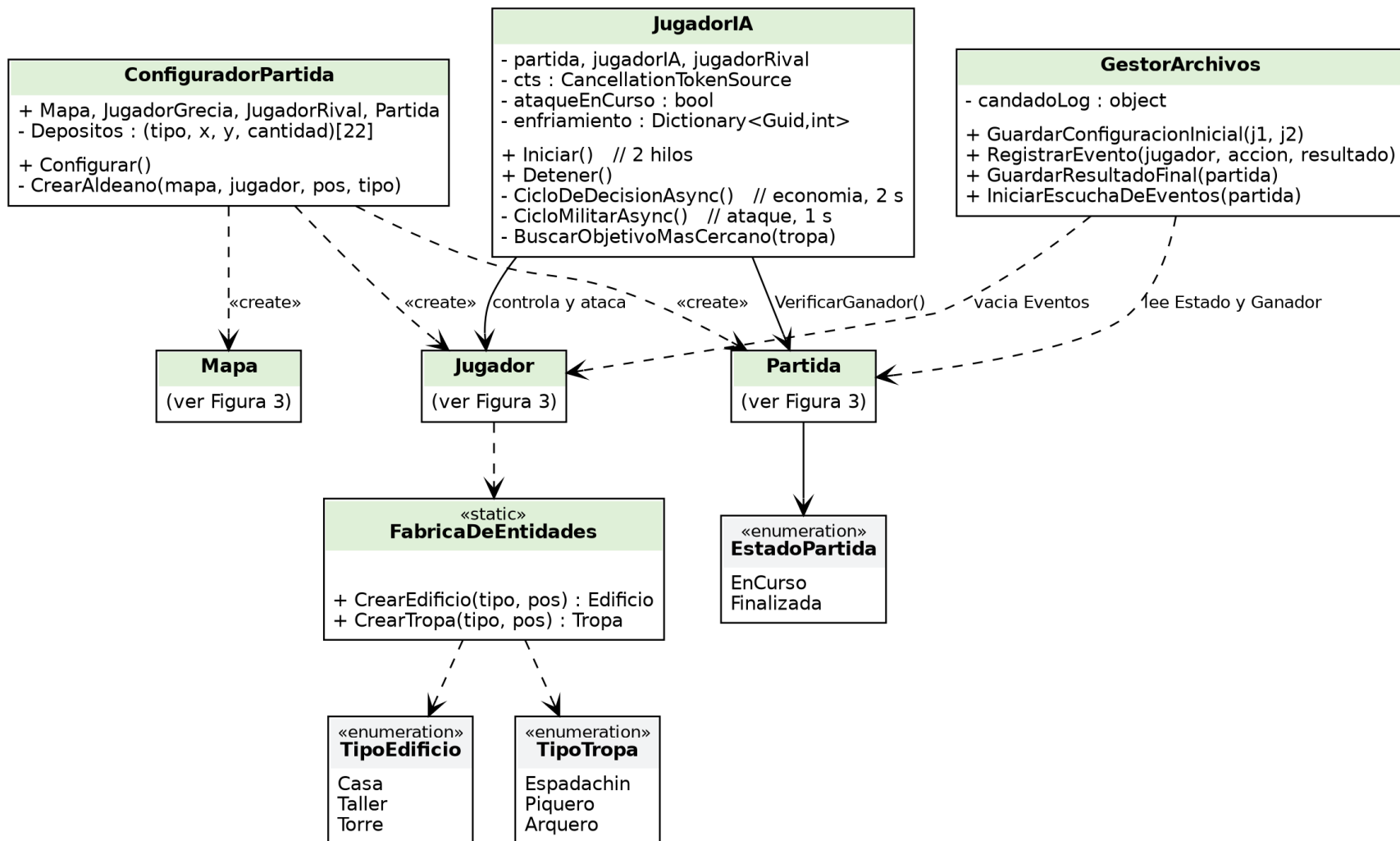
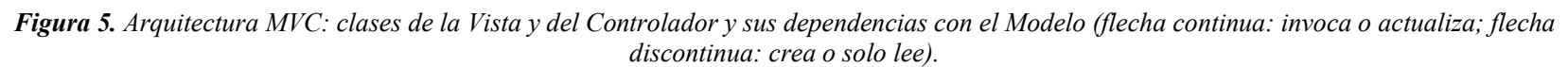


Figura 4. Diagrama de clases del Modelo: servicios, IA y persistencia.



7.4 Diagramas de flujo de las actividades concurrentes

Las Figuras 6, 7 y 8 detallan tres actividades: una orden del jugador que termina en un hilo de construcción, el ciclo del hilo de un aldeano que compite por los recursos, y los dos hilos de la IA. Cada figura se divide en dos paneles, (a) y (b), que se conectan mediante los círculos A y B.

Orden del jugador: selección y orden

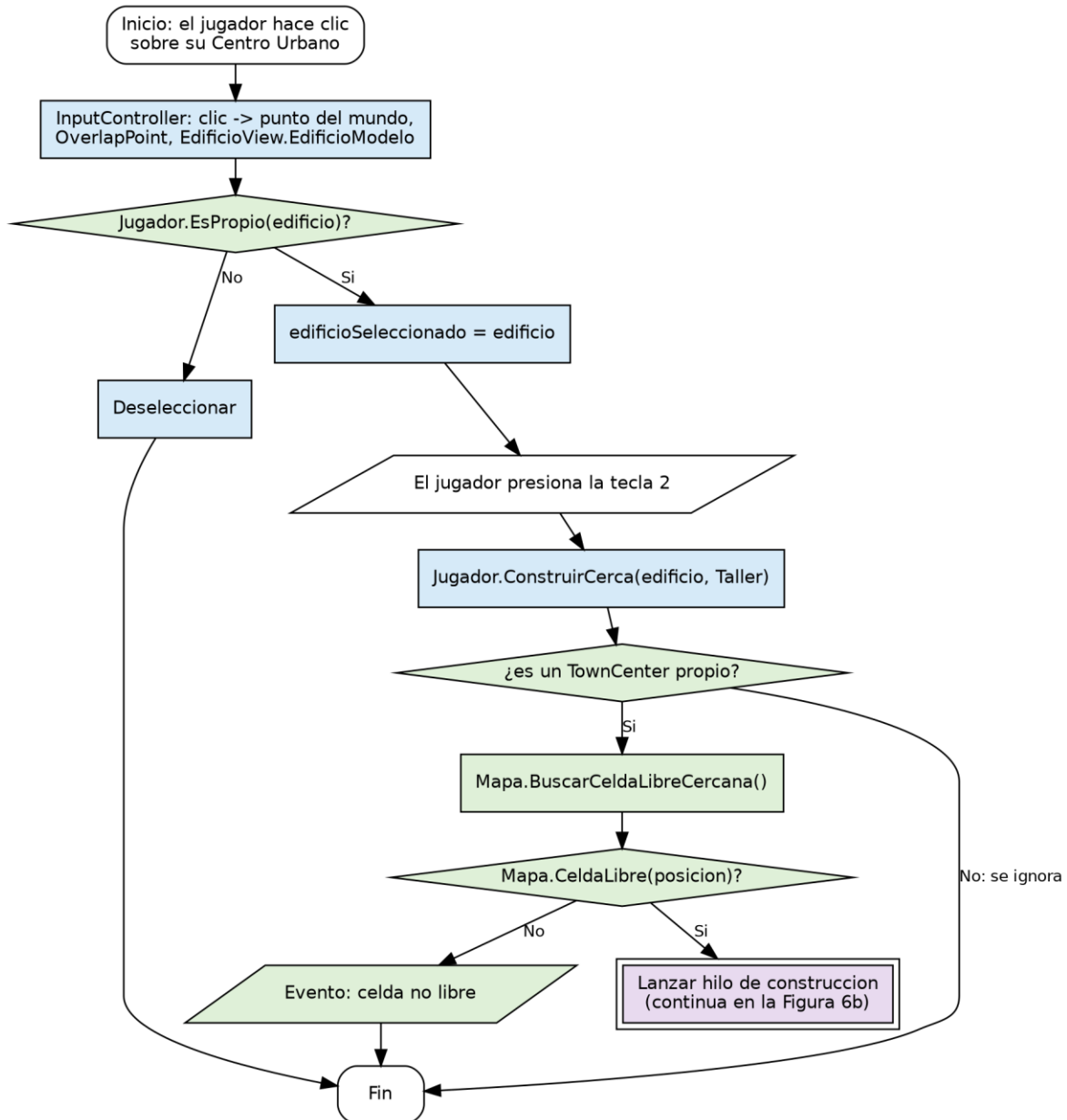


Figura 6a. Orden del jugador (clic en el Centro Urbano y tecla 2): (a) selección y orden.

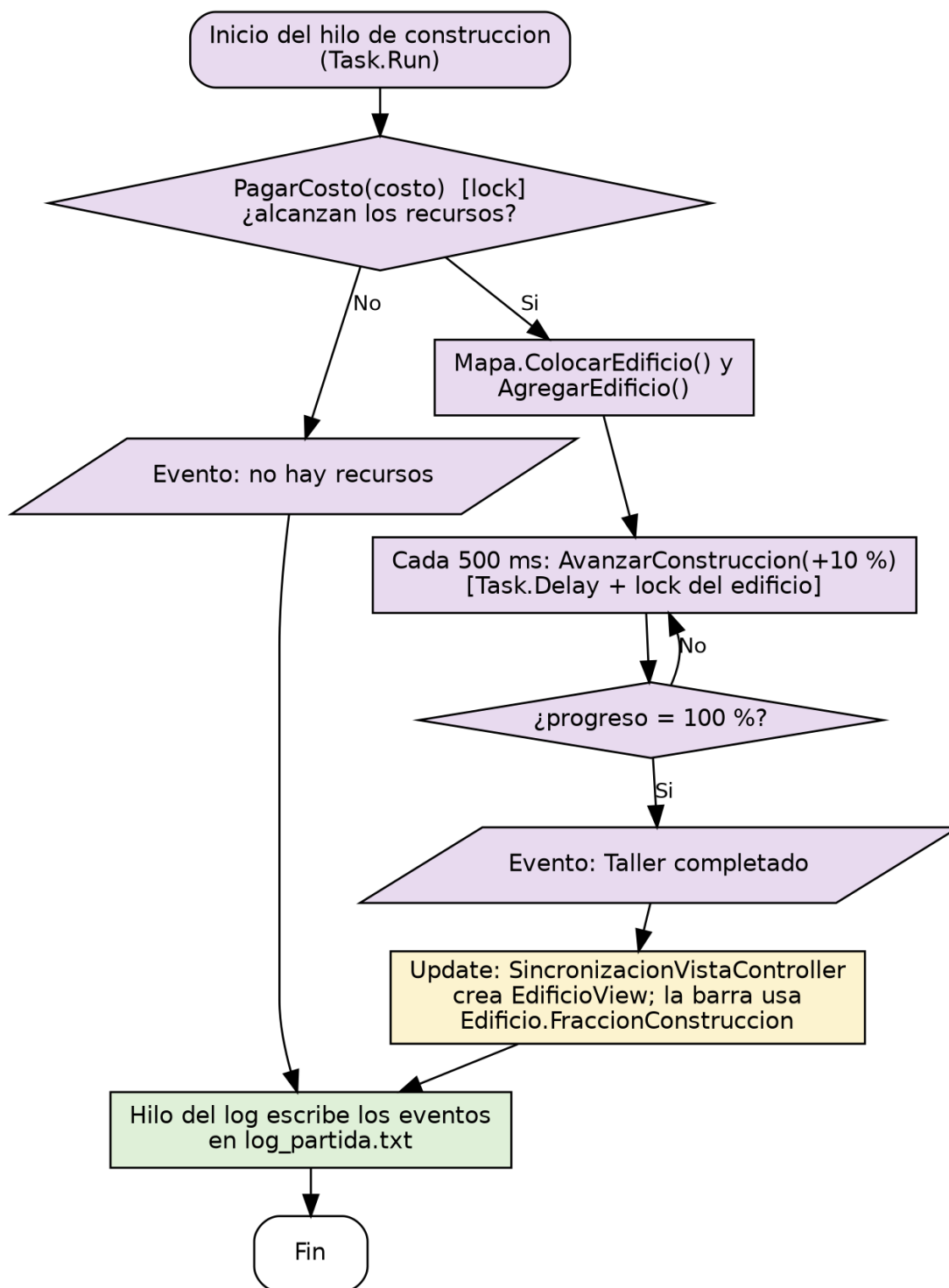
Orden del jugador: hilo de construcción

Figura 6b. Orden del jugador: (b) hilo de construcción.

Hilo del aldeano: elección del depósito y desplazamiento

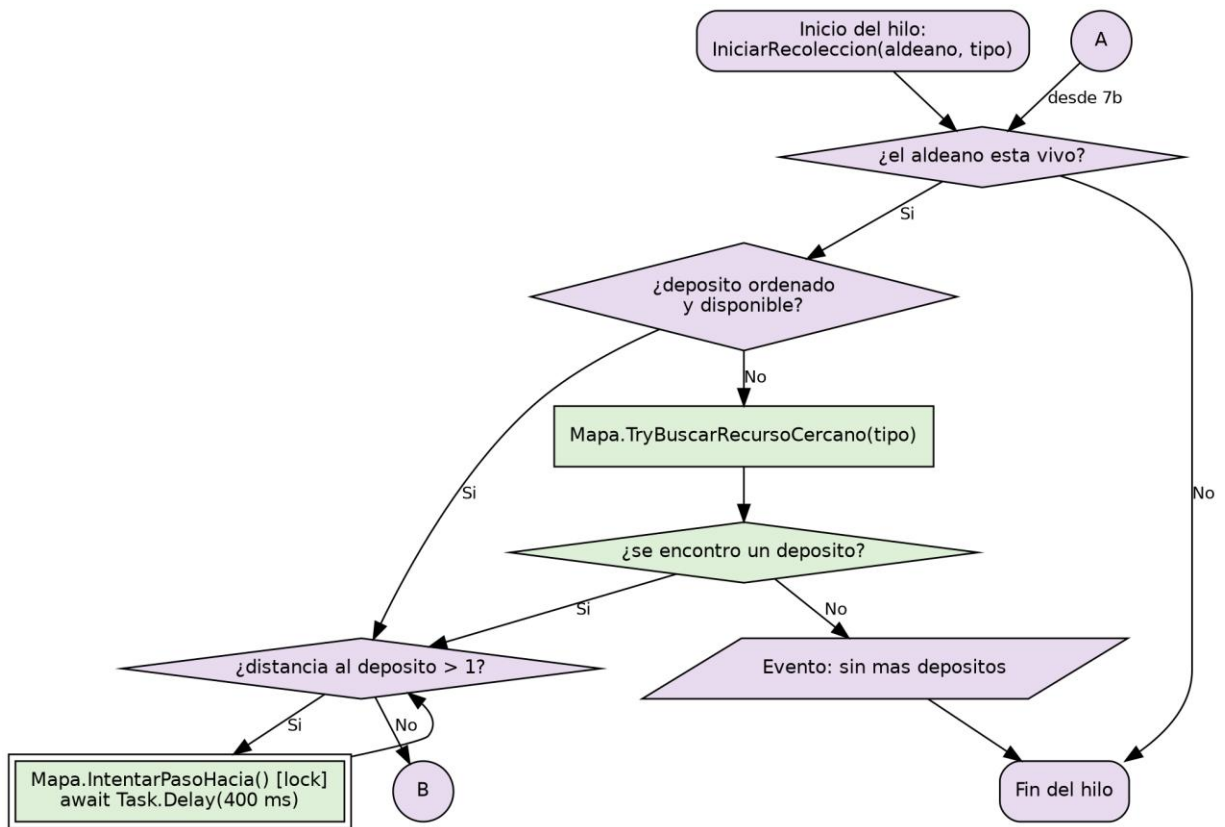


Figura 7a. Hilo de recolección de un aldeano: (a) elección del depósito y desplazamiento.

Hilo del aldeano: recolección y relevo

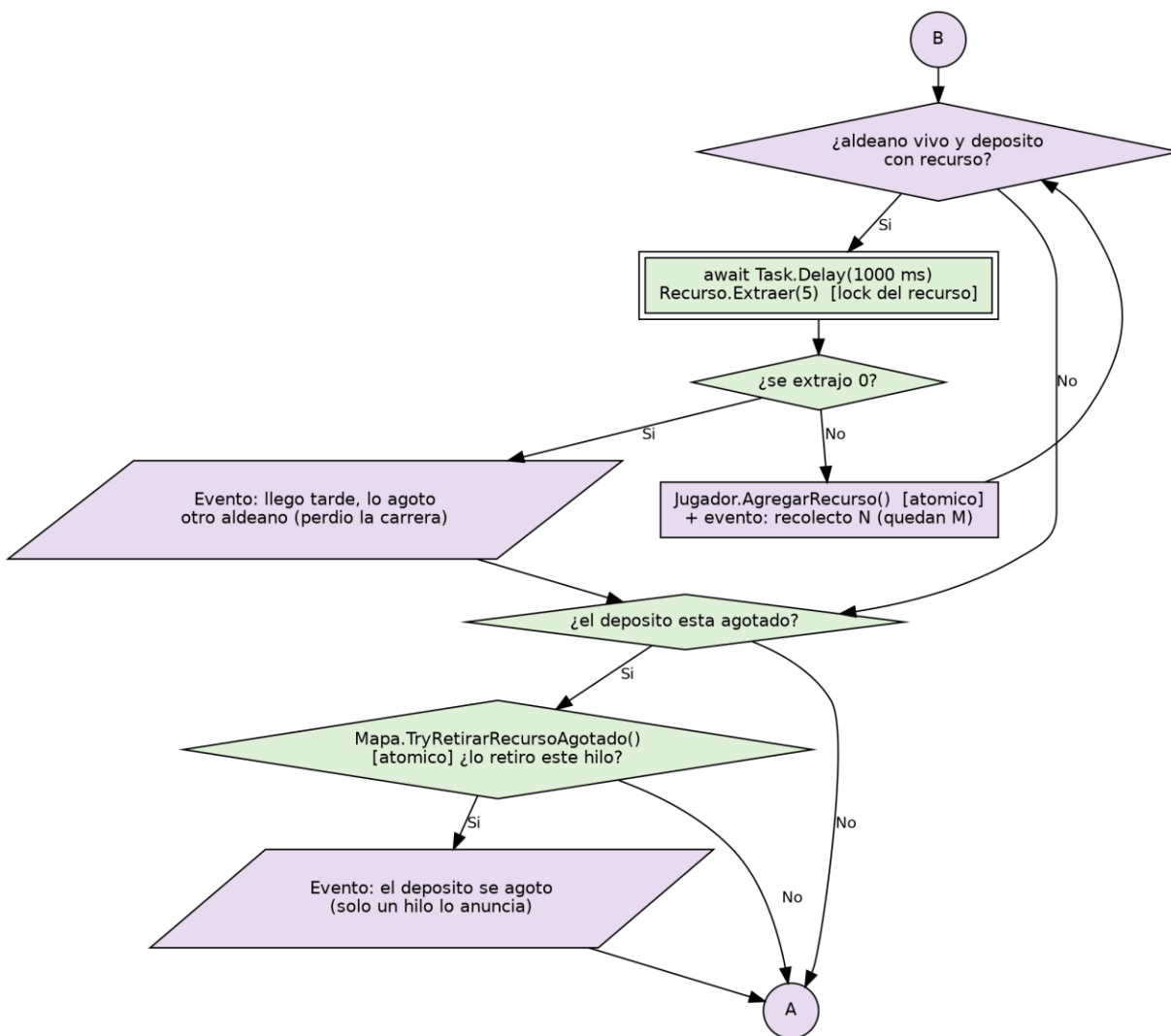
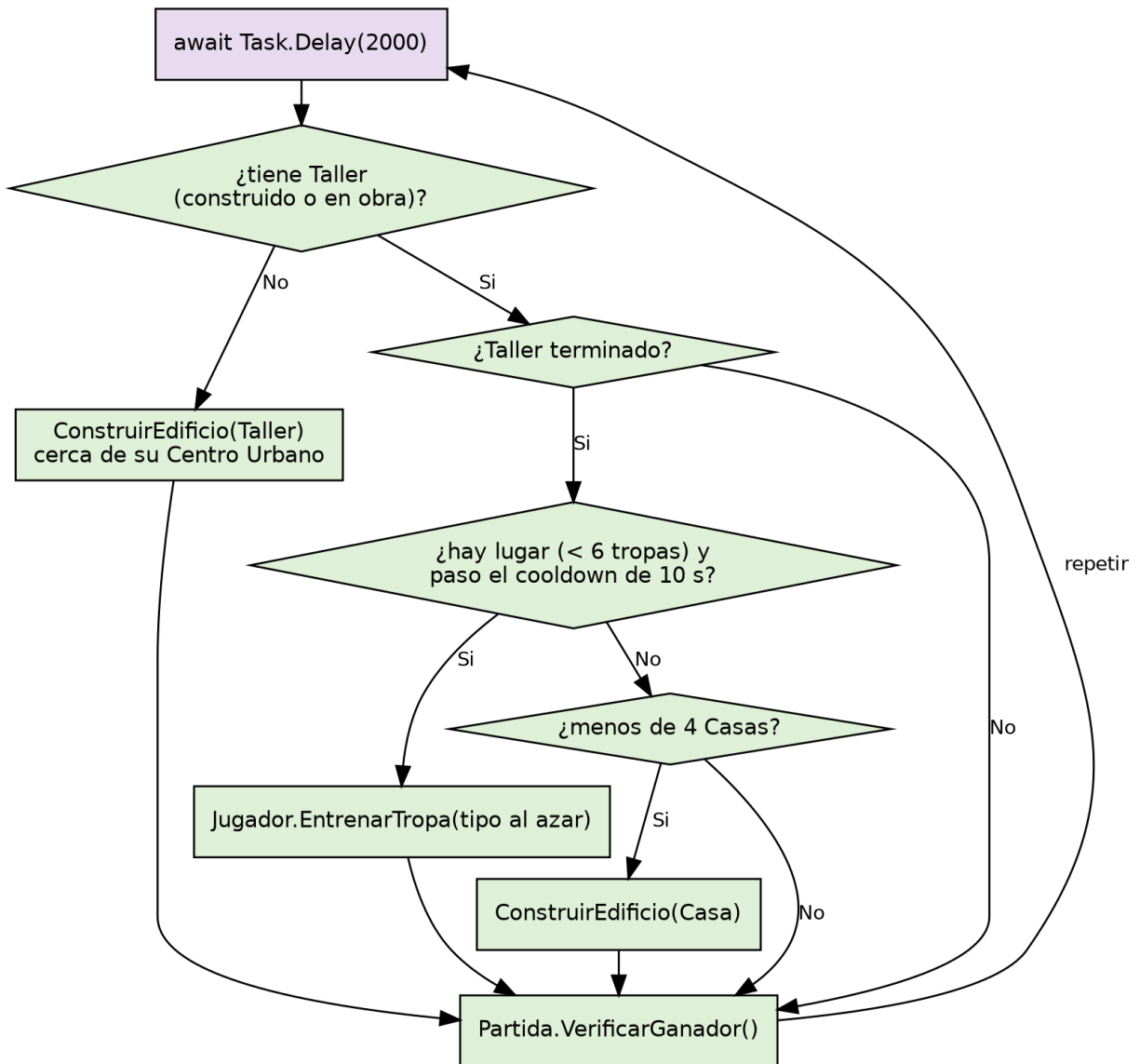


Figura 7b. Hilo de recolección de un aldeano: (b) recolección y relevo. En cualquier `Task.Delay`, si se cancela el token (orden nueva o `DetenerHilos`), el hilo termina.

IA: ciclo de economía**Figura 8a.** Hilos de la IA: (a) ciclo de economía.

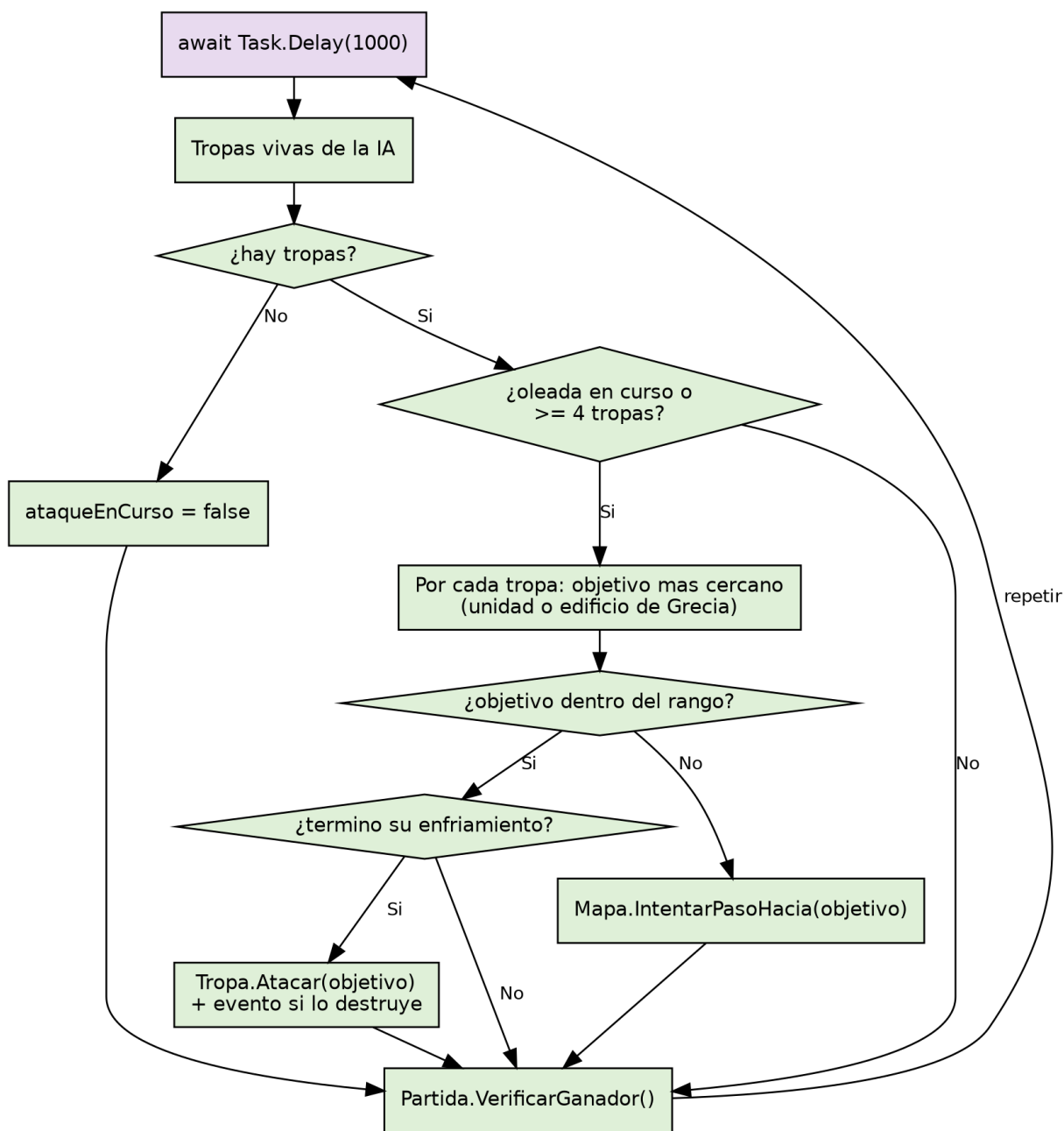
IA: ciclo militar

Figura 8b. Hilos de la IA: (b) ciclo militar. Ambos hilos terminan cuando la partida está finalizada o se cancela su token.

8. Pruebas de escritorio

8.1 Metodología y alcance

Las pruebas verifican el flujo y el comportamiento de las secciones más críticas: concurrencia, ataques y condición de victoria. Cada caso tiene un identificador, las condiciones de entrada, el resultado esperado y el resultado obtenido. Los casos se ejecutaron con un programa de prueba que instancia las clases reales del Modelo, con el `Recurso.cs` y el `TipoRecurso.cs` del proyecto, y registra los valores realmente obtenidos: la columna «Resultado obtenido» no es una estimación sino la salida de la ejecución.

Alcance y límites. Las pruebas se ejecutaron sobre el Modelo, sin el motor Unity. Por eso no cubren clics, colliders, animaciones ni el aspecto en pantalla; esos aspectos se verifican con las pruebas manuales de la sección 8.6. La comunicación en red no se prueba porque no fue implementada (sección 5). **Entorno:** .NET 8 con el lenguaje fijado en C# 9 (versión del compilador de Unity 6), sobre Linux.

8.2 Traza de escritorio de la condición de carrera

Antes de los resultados automatizados se presenta la traza, paso a paso, de la situación que el `lock` de `Recurso.Extraer` evita. Dos aldeanos, uno de cada bando, piden 8 unidades a un depósito que solo conserva 10.

Tabla 13. Traza sin sincronización (situación que se evita).

Paso	Hilo	Acción	Cantidad del depósito
1	Grecia	Lee Cantidad = 10 y calcula <code>extraido = 8</code>	10
2	IA	Lee Cantidad = 10 (Grecia aún no restó) y calcula <code>extraido = 8</code>	10
3	Grecia	<code>Cantidad = 10 - 8</code>	2
4	IA	<code>Cantidad = 2 - 8</code>	-6
Resultado	—	Se entregaron 16 unidades de un depósito de 10 y el saldo quedó negativo	-6

Tabla 14. Traza con lock (comportamiento real de `Recurso.Extraer`).

Paso	Hilo	Acción	Cantidad del depósito
1	Grecia	Entra al <code>lock</code>	10
2	IA	Intenta entrar: queda esperando	10
3	Grecia	<code>extraido = mín(8, 10) = 8</code> ; Cantidad = 2; sale del <code>lock</code> y devuelve 8	2
4	IA	Entra al <code>lock</code>	2
5	IA	<code>extraido = mín(8, 2) = 2</code> ; Cantidad = 0; sale y devuelve 2	0
Resultado	—	$8 + 2 = 10$, que coincide con lo que había. En su siguiente intento la IA obtiene 0 y registra «llegó tarde»	0

8.3 Casos de concurrencia

Tabla 15. Casos de prueba de concurrencia.

ID	Descripción	Entrada / condiciones	Resultado esperado	Resultado obtenido	Estado
PE-C1	Dos aldeanos de bandos rivales compiten por el mismo depósito	Depósito de Madera con 60 unidades en (10,10). Aldeano de Grecia en (8,10) y de la IA en (12,10), ambos asignados a Madera.	Lo recolectado entre ambos suma exactamente 60; el agotamiento se anuncia una sola vez; la celda queda libre.	Grecia 30 + IA 30 = 60; avisos de agotamiento = 1; celda libre = True	Aprobado
PE-C2	Seis aldeanos (3 por bando) compiten por tres depósitos	3 depósitos de Madera de 100 en (9,9), (10,10) y (9,10). 3 aldeanos de Grecia y 3 de la IA.	Lo recolectado suma exactamente 300; 3 avisos de agotamiento (uno por depósito); ningún hilo reporta errores.	total recolectado = 300; avisos de agotamiento = 3; errores de hilos = no	Aprobado
PE-C3	Pago de costos atómico (condición de carrera clásica 'leer, modificar, escribir')	50 rondas; en cada una, 10 hilos simultáneos llaman PagarCosto(30 de Oro) con 100 de Oro disponibles.	En cada ronda exactamente 3 pagos exitosos y saldo final de 10 Oro (nunca negativo).	rondas incorrectas = 0 de 50; última ronda: 3 pagos, Oro final 10	Aprobado
PE-C4	Entrenamiento en paralelo: cada tropa del lote ocupa su propia celda	Taller construido; EntrenarLote de 5 Arqueros (20 de Oro c/u) con 100 de Oro, punto de despliegue (5,5).	5 tropas desplegadas en 5 celdas distintas (sin apilarse), Oro final 0.	tropas = 5; celdas distintas = 5; Oro final = 0	Aprobado
PE-C5	Una sola orden activa por unidad (la orden nueva cancela la anterior)	Aldeano recolectando Madera; se le ordena caminar a la celda vacía (15,15).	Llega a (15,15) y la Madera deja de aumentar.	llegó a (15,15) = True; Madera al llegar = 110; 2,5 s después = 110	Aprobado
PE-C6	Cancelación ordenada de todos los hilos (Jugador.DetenerHilos)	Aldeano recolectando; se invoca DetenerHilos().	Tras la cancelación, la Madera deja de aumentar.	Madera tras cancelar = 110; 2,5 s después = 110	Aprobado
PE-C7	Cooldown de entrenamiento (regla del Modelo, con lock)	Taller propio construido; cooldown de 8 s. Dos órdenes de entrenamiento seguidas y una tercera a los 8,3 s.	1. ^a aceptada, 2. ^a rechazada por enfriamiento, 3. ^a aceptada; EntrenamientoDisponible pasa de true a false y de nuevo a true.	disponible True->False; órdenes: True, False, True; faltaban 8.0 s; disponible tras 8,3 s = True	Aprobado
PE-C8	Competencia en la partida real (configuración inicial completa)	ConfiguradorPartida.Configurar(): 22 depósitos y 3 aldeanos por bando; se observa durante 15 s.	Los aldeanos caminan (cambian de posición), Grecia acumula Madera y no hay errores de hilos.	depósitos = 22; aldeanos que caminaron = 3/3; Madera 160, Oro 150, Piedra 155; errores = no	Aprobado

8.4 Casos de ataques

Tabla 16. Casos de prueba de ataques y validaciones.

ID	Descripción	Entrada / condiciones	Resultado esperado	Resultado obtenido	Estado
PE-A1	Ataque cuerpo a cuerpo dentro del rango	Espadachín de Grecia en (5,5) ataca a un Espadachín rival en (6,5) (distancia 1, rango 1, daño 10).	El ataque se aplica (true) y la vida del objetivo baja de 50 a 40; se registra el evento.	resultado = True; vida objetivo = 40/50; eventos 'atacó' = 1	Aprobado
PE-A2	Ataque fuera de rango (se rechaza)	Espadachín (rango 1) en (5,5) ataca a un objetivo en (8,5) (distancia 3).	Se rechaza (false), la vida no cambia y queda el evento 'fuera de rango'.	resultado = False; vida objetivo = 50/50; evento 'fuera de rango' = 1	Aprobado
PE-A3	Ataque a distancia (Arquero, rango 3)	Arquero en (5,5). Objetivo A a distancia 3 (8,5); objetivo B a distancia 4 (9,5).	Golpea al objetivo A (daño 8) y no puede golpear al B.	A (dist. 3): True, vida 42/50; B (dist. 4): False, vida 50/50	Aprobado

ID	Descripción	Entrada / condiciones	Resultado esperado	Resultado obtenido	Estado
PE-A4	Dstruccion de una unidad: libera su celda y se registra el evento	Espadachín de Grecia golpea 5 veces a un Espadachín rival (vida 50, daño 10).	El objetivo muere, su celda queda libre en el Mapa y queda el evento 'destruyó'.	celda libre antes = False; objetivo vivo = False; celda libre después = True; eventos 'destruyó' = 1	Aprobado
PE-A5	Dstruccion de un edificio: libera su celda	Espadachín de Grecia golpea 10 veces una Casa rival (vida 100, daño 10).	La Casa queda destruida y su celda libre.	casa destruida = True; celda libre = True; vida = 0/100	Aprobado
PE-A6	La Torre dispara sola (hilo de defensa) sin intervención del Controlador	Centro Urbano de Grecia; ConstruirCerca(Torre) (80 Piedra + 40 Metal); un Espadachín rival dentro del rango 3 de la Torre.	La Torre se construye, se activa sola y dispara: la vida del enemigo baja (8 por disparo).	torre construida = True; vida del enemigo = 34/50 (2 disparos); costo pagado: Piedra 20, Metal 60	Aprobado
PE-A7	Validaciones: no se puede ordenar con unidades ajenas ni a un objetivo nulo	Grecia intenta ordenar un ataque con una tropa del rival, y otro ataque con objetivo nulo.	Ambas órdenes se rechazan (false) y no cambia ninguna vida.	ataque con tropa ajena = False; con objetivo nulo = False; vida de la tropa de Grecia = 50/50	Aprobado
PE-A8	Movimiento: destino fuera del mapa y unidad ajena	Orden de movimiento a (-9,-9) con una unidad propia, y a (5,5) con una unidad del rival.	Ambas se ignoran: las posiciones no cambian.	unidad propia en (5,5); unidad ajena en (12,12)	Aprobado

8.5 Casos de victoria y archivos

Tabla 17. Casos de prueba de la condición de victoria y de los archivos.

ID	Descripción	Entrada / condiciones	Resultado esperado	Resultado obtenido	Estado
PE-V1	Victoria de Grecia: se destruye el Centro Urbano de la IA	Partida en curso; el Centro Urbano de la IA (500 de vida) recibe 500 de daño.	VerificarGanador = true; Estado = Finalizada; Ganador = 'Grecia'; GanoJugador(Grecia) = true y GanoJugador(IA) = false.	antes del golpe = False; después = True; Estado = Finalizada; Ganador = Grecia; GanoJugador(Grecia) = True, GanoJugador(IA) = False	Aprobado
PE-V2	Derrota de Grecia: la IA destruye su Centro Urbano	El Centro Urbano de Grecia recibe 500 de daño.	Ganador = 'El Resto'; GanoJugador(Grecia) = false (el Controlador muestra el panel de derrota).	fin = True; Ganador = El Resto; GanoJugador(Grecia) = False	Aprobado
PE-V3	Derrota por pérdida de todas las unidades militares (segunda condición de la guía)	Grecia entrena una tropa y la pierde; su Centro Urbano sigue intacto.	Grecia está derrotada aunque su Centro Urbano sigue en pie: Ganador = 'El Resto'.	con la tropa viva = False; tras perderla = True; Ganador = El Resto; Centro Urbano de Grecia destruido = False	Aprobado
PE-V4	Un jugador que nunca tuvo tropas no se considera derrotado	Inicio de la partida: ninguno de los dos jugadores ha entrenado tropas.	VerificarGanador = false; Estado = EnCurso; Ganador nulo.	fin = False; Estado = EnCurso; Ganador = null	Aprobado
PE-V5	Empate: ambos Centros Urbanos caen a la vez	Los dos Centros Urbanos reciben 500 de daño antes de verificar.	Ganador = 'Empate'; GanoJugador es false para ambos.	Ganador = Empate; GanoJugador(Grecia) = False, GanoJugador(IA) = False	Aprobado
PE-V6	Idempotencia: un resultado ya anunciado no se pisa	Grecia gana; después también cae el Centro Urbano de Grecia y se vuelve a verificar.	El Ganador sigue siendo 'Grecia' y VerificarGanador devuelve true.	segunda verificación = True; Ganador = Grecia	Aprobado

ID	Descripción	Entrada / condiciones	Resultado esperado	Resultado obtenido	Estado
PE-V7	Verificación concurrente: 20 hilos verifican a la vez tras la caída del Centro Urbano	Centro Urbano de la IA destruido; 20 hilos llaman VerificarGanador() al mismo tiempo.	Un único evento 'FinDePartida' en la cola de la Partida y Ganador consistente.	eventos FinDePartida = 1; Ganador = Grecia	Aprobado
PE-V8	resultado_final.txt lo escribe GestorArchivos al terminar la partida	Partida con hilo del log activo; cae el Centro Urbano de la IA; nadie llama a GuardarResultadoFinal desde fuera.	Aparece resultado_final.txt con 'Ganador: Grecia' y el log contiene 'Resultado final: Grecia'.	existía antes = False; resultado_final.txt contiene 'Ganador: Grecia' = True; log contiene 'Resultado final: Grecia' = True	Aprobado
PE-V9	Cierre del juego con la partida en curso: no se escribe resultado	Se inicia el hilo del log y se cancela con la partida sin terminar.	No existe resultado_final.txt.	existe resultado_final.txt = False	Aprobado

Resumen: 25 de 25 casos aprobados (8 de 8 de concurrencia, 8 de 8 de ataques y 9 de 9 de victoria y archivos).

9. Conclusiones

9.1 Cumplimiento de los requisitos

Tabla 19. Cumplimiento de los requisitos de la guía.

Requisito de la guía	Estado	Evidencia
Interfaz gráfica de escritorio en Unity (sin consola ni WebGL)	Implementada; falta generar y probar el build de escritorio	Escena y prefabs de Unity; el build se genera con File > Build Settings
Patrón MVC con separación clara	Cumplido	Secciones 3 y 7; auditoría de la sección 3.4
Concurrencia con hilos y sincronización	Cumplido	Sección 4; casos PE-C1 a PE-C8
Comunicación en red	No implementada (opcional por indicación de la docente)	Sección 5
Manejo de archivos: configuración, log y resultado	Cumplido	Sección 6; casos PE-V8 y PE-V9
Colecciones para unidades y edificios	Cumplido	Dictionary, ConcurrentDictionary y ConcurrentQueue (sección 4.3)
Validación de las acciones del usuario	Cumplido	Jugador valida propiedad, celda, rango, costos y límites (PE-A2, PE-A7 y PE-A8)
Manejo de excepciones (try-catch)	Cumplido	Jugador.Lanzar, ciclos de la IA y GestorArchivos (IOException)
Diagramas UML y de flujo	Cumplido	Sección 7
Pruebas de escritorio	Cumplido en las 25 automatizadas; manuales por completar	Sección 8

9.2 Limitaciones

- No se implementó la comunicación en red (sección 5).

- El límite de población de las Casas está definido pero no se aplica, el contador de población del HUD no se actualiza y los aldeanos no se pueden entrenar.
- La selección de unidades no tiene resaltado visual y las teclas disponibles no se muestran en pantalla.
- El movimiento de aldeanos y tropas usa la celda vecina más cercana al destino, sin búsqueda de caminos, y puede trabarse ante obstáculos.
- Las pruebas automatizadas cubren el Modelo; la interfaz se verifica de forma manual.
- Está pendiente generar y probar el build de escritorio.

9.3 Trabajo futuro

- Implementar la comunicación en red según la propuesta de la sección 5.3.
- Permitir entrenar aldeanos y aplicar el tope de población que otorgan las Casas.
- Resaltar la selección y mostrar en pantalla las acciones disponibles.
- Incorporar búsqueda de caminos (A*) para aldeanos y tropas.
- Parametrizar la dificultad de la IA y ampliar los tipos de tropa y de edificio.

9.4 Aprendizajes

El proyecto mostró que la concurrencia se apoya en decisiones pequeñas y sistemáticas: esperar con `Task.Delay` en lugar de bloquear hilos, proteger cada dato compartido con el mecanismo adecuado, tomar los candados siempre en el mismo orden, disparar los eventos fuera de los candados y hacer que todo hilo pueda cancelarse. También mostró que la separación MVC no se consigue en la primera versión: fue necesaria una auditoría posterior, con un criterio claro, para encontrar y trasladar al Modelo la lógica que se había filtrado a la Vista y al Controlador.

10. Referencias

- Krasner, G. E., y Pope, S. T. (1988). A cookbook for using the model-view controller user interface paradigm in Smalltalk-80. *Journal of Object-Oriented Programming*, 1(3), 26–49.
- Microsoft. (s. f.). *Cancellation in managed threads*. Learn.microsoft.com.
<https://learn.microsoft.com/dotnet/standard/threading/cancellation-in-managed-threads>
- Microsoft. (s. f.). *lock statement*. Learn.microsoft.com.
<https://learn.microsoft.com/dotnet/csharp/language-reference/statements/lock>
- Microsoft. (s. f.). *Managed threading (C#)*. Learn.microsoft.com.
<https://learn.microsoft.com/dotnet/standard/threading/managed-threading-basics>
- Microsoft. (s. f.). *Sockets (System.Net.Sockets)*. Learn.microsoft.com.
<https://learn.microsoft.com/dotnet/api/system.net.sockets>

- Microsoft. (s. f.). *Task-based asynchronous programming*. Learn.microsoft.com.
<https://learn.microsoft.com/dotnet/standard/parallel-programming/task-based-asynchronous-programming>
- Microsoft. (s. f.). *Thread-safe collections*. Learn.microsoft.com.
<https://learn.microsoft.com/dotnet/standard/collections/thread-safe/>
- Unity Technologies. (s. f.). *Unity manual: Multithreading and native plug-ins*. Docs.unity3d.com.
<https://docs.unity3d.com/Manual/UnderstandingPerformanceThreading.html>